

Learning-Augmented B-Trees

Xinyuan Cao
Georgia Institute of Technology
xcao78@gatech.edu

Jingbang Chen
University of Waterloo
j293chen@uwaterloo.ca

Li Chen
Georgia Institute of Technology
lichen@gatech.edu

Chris Lambert
Carnegie Mellon University
cslamber@andrew.cmu.edu

Richard Peng
University of Waterloo
y5peng@uwaterloo.ca

Daniel Sleator
Carnegie Mellon University
sleator@cmu.edu

Abstract

We study learning-augmented binary search trees (BSTs) and B-Trees via Treaps with composite priorities. The result is a simple search tree where the depth of each item is determined by its predicted weight w_x . To achieve the result, each item x has its composite priority $-\lceil \log \log(1/w_x) \rceil + U(0, 1)$ where $U(0, 1)$ is the uniform random variable. This generalizes the recent learning-augmented BSTs [Lin-Luo-Woodruff ICML'22], which only work for Zipfian distributions, to arbitrary inputs and predictions. It also gives the first B-Tree data structure that can provably take advantage of localities in the access sequence via online self-reorganization. The data structure is robust to prediction errors and handles insertions, deletions, as well as prediction updates.

1 Introduction

The development of machine learning has sparked significant interest in its potential to enhance traditional data structures. First proposed by Kraska et al. [KBCDP18], the notion of *learned index* has gained much attention since then [KBCDP18; DMYWDLZCGK+20; FV20]. Algorithms with predictions have also been developed for an increasingly wide range of problems, including shortest path [CSVZ22], network flow [PZ22; LMRX20], matching [CSVZ22; DILMV21; CI21], spanning tree [ELMS22], and triangles/cycles counting [CEILNRSWWZ22], with the goal of obtaining algorithms that get near-optimal performances when the predictions are good, but also recover prediction-less worst-case behavior when predictions have large errors [MV20].

Regarding the original learned index question, which uses learning to speed up search trees, developing data structures optimal to the input sequence has been extensively studied in the field of data structures. Melhorn [Meh75a] showed that a nearly optimal static tree can be constructed in linear time when estimates of key frequencies are provided. Extensive work on this topic culminated in the study of dynamic optimality, where tree balancing algorithms (e.g. splay trees [ST85b], Tango trees [DHIP07]), as well as lower bounds (e.g. Interleave lower bounds) are conjectured to be within constant factors of optimal.

This paper examines the performance of learning augmented search trees in both static and dynamic settings. Specifically, we show how to incorporate learned advice to build both near-optimal

static search trees, as well as dynamic search trees whose performances match the working set bound of the access sequence. We start by presenting a composite priority function that integrates learned advice into Treaps, a family of binary search trees. Given an oracle that can predict the number of occurrences of each item (up to a generous amount of error), the tree produced via these composite priorities is within constants of the static optimal ones. (Section 4). A major advantage of this priority-based approach is it naturally extends to B-Trees via B-Treaps [Gol09], leading to the first B-Tree with access-sequence-dependent performance bounds.

We then study these composite priorities based Treaps in the dynamic setting, where the tree can undergo changes after each access. We show that given a learned oracle can correctly predict the time interval until the next access, B-Treaps with composite priorities can achieve the working set bound (Section 6). This bound can be viewed as a strengthening of the static optimality/entropy bound that takes temporal locality of keys into account. Previous analyses of B-Trees only focus on showing an $O(\log_B n)$ cost per access in the worst case. (Section 5) Finally, we show that our data structure is robust to prediction errors (Section 6). That is, the performances of B-Treaps with composite priorities degrade smoothly with the mean absolute error between the generated priorities and the ground truth priorities.

The composite priority function takes the learned advice and perturbs it with a small random value. The depth of each item in the tree is related to the learned advice and is worst-case $O(\log n)$ and $O(\log_B n)$ in the BST and the B-Tree case. This generalizes the result of [LLW22] and removes their assumption on the inputs. On the other hand, in the setting of dynamic predictions, our data structure achieves bounds unknown to any existing binary search trees.

2 Overview

In this section, we give a brief overview of our data structures and analyses. Our starting point is the learning-augmented Treap data structure from Lin, Luo, and Woodruff [LLW22]. They directly incorporated predictions of node frequencies as the node priorities in the Treap data structure.

Learning Augmented Treaps via Composite Priority Functions Treap is a tree-balancing mechanism initially designed around randomized priorities [AS89]. One of its central observations is that requiring the heap priority among unique node priorities gives a unique tree: the root must be the one with maximum priority, after which both its left and right subtrees are also unique by a recursive/inductive application of this reasoning. Aragon and Siedel show that when a node's priority is modified, the new unique tree state can be obtained by rotations along the current path [AS89], and such a rebuilding can also be done recursively/functionally [BR98]. Therefore, the cost of accessing this Treap is precisely the depth of the accessed node in the unique tree obtained.

When the priorities are assigned randomly, the resulting tree is balanced with high probability. Intuitively, this is because the root is likely to be picked among the middle elements. However, if some node is accessed very frequently (e.g. 10% of the time), it's natural to assign it a larger priority. Therefore, setting the priority to a function of access frequencies, as in [LLW22], is a natural way to obtain an algorithm whose performances are better on more skewed access patterns. However, when the priority is set directly to access frequency, the algorithm does not degrade smoothly to $O(\log n)$: if elements $1, 2, \dots, \sqrt{n}$ are accessed with frequency $\sqrt{n}, \sqrt{n} - 1, \dots, 1$, setting priorities as such will result in a path of $\Theta(\sqrt{n})$. Partly as a result of this, the analysis in [LLW22] was limited only to when frequencies are from the Zipfian distribution.

Building upon these ideas, we introduce the notion of a composite priority function, and choose

it to be a mixture of the randomized priority function from [AS89] and the frequency-based priority from [LLW22]. This function can be viewed as adding back a uniformly random adjustment between $[0, 1]$ to the resulting priorities to ensure that nodes with the same priorities are still balanced as in worst-case Treaps.

However, for keys with large access frequencies, e.g. $n^{1/2}$ vs $\frac{1}{2}n^{1/2}$, the additive $[0, 1]$ is not able to offset differences between them. Therefore, we need to reduce the range of the priorities so that this $[0, 1]$ is still effective in handling nodes with similar priorities. Here, a natural candidate is to take the logarithm of the predicted frequencies. However, we show in Section 4.4 that this is also problematic for obtaining a statically optimal tree. Instead, we obtain the desired properties by taking logs once again. Specifically, we show in Theorem 4.4 that by setting the priority to

$$-\left\lfloor \log \log \frac{1}{w_x} \right\rfloor + U(0, 1),$$

the expected depth of node x is $O(\log(1/w_x))$.

One way to intuitively understand the choice of this composite priority scheme is to view it as a bucketing scheme on $\log(1/w_x)$, and apply the intuition that a randomized Treap over a set of m items has height $O(\log m)$. Let S_t to be the set of elements such that $\lfloor \log \log(1/w_x) \rfloor = t$, every tree path towards the root has $O(\log |S_t|)$ elements in S_t in expectation. Since the total weights $\sum_x w_x \leq 1$, we can bound the size of S_t by $|S_t| \leq 2^{2^{t+1}}$ and thus $\log |S_t| \leq 2^{t+1}$. Moreover, every x -to-the-root path only consists of nodes in $S_t, S_{t-1}, \dots, S_0$ and we can bound the expected depth by

$$\log |S_t| + \log |S_{t-1}| + \dots + \log |S_0| \leq 2^{t+1} + 2^t + \dots + 2^1 \leq 2^{t+2} \leq 4 \log(1/w_x)$$

up to a constant factor. We discuss counterexamples to other ways of setting priorities in Section 4.4.

Static Learning Augmented B-Trees Our Treap-based scheme generalizes to B-Trees, where each node has B instead of 2 children. These trees are highly important in external memory systems due to the behavior of cache performances: accessing a block of B entries has a cost comparable to the cost of accessing $O(1)$ entries.

By combining the B-Treaps by Golovin [Gol09] with the composite priorities, we introduce a new learning-augmented B-Tree that achieves similar bounds under the *External Memory Model*. We show in Theorem 5.2 that for any weights over elements $\mathbf{w}$, by setting the priority to

$$-\lfloor \log_4 \log_B(1/w_x) \rfloor + U(0, 1),$$

the expected depth of node x is $O(\log_B(1/w_x))$.

This gives the first optimal static B-Trees if we set $\mathbf{w}$ to be the marginal distribution of elements in the access sequence. That is, if we know $f_1, f_2, \dots, f_n$ the frequencies of each element that appears in the access sequence, we can build a static B-Tree such that the total number of tree nodes touched is roughly

$$\sum_{i \in [n]} f_i \log_B \frac{m}{f_i}$$

where $m = \sum_i f_i$, the length of the access sequence.

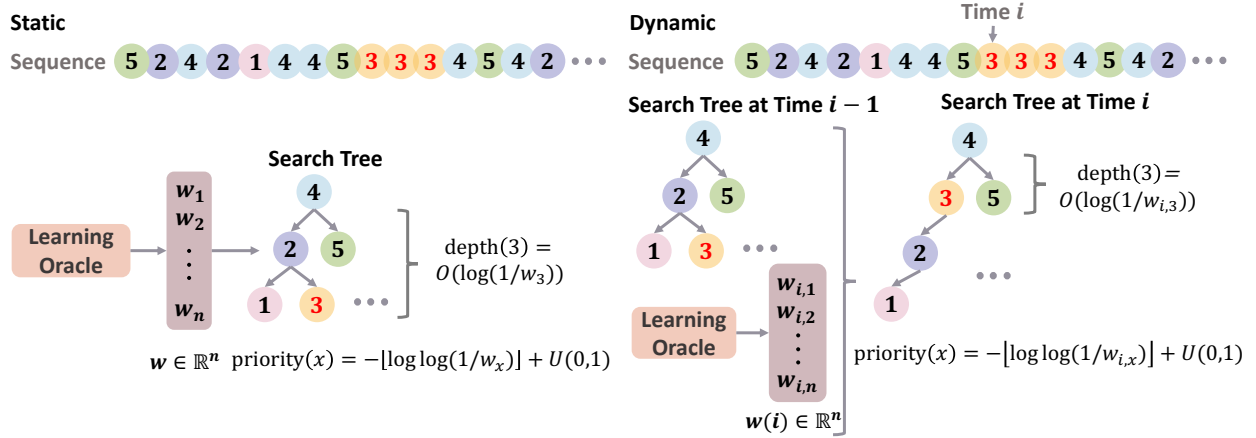


Figure 1: Sketch for static and dynamic learning augmented B-Trees. Since item 3 has a higher frequency around time i , the dynamic B-Trees adjust the priority accordingly.

Dynamic Learning Augmented B-Trees We also consider the dynamic setting in which we continually update the priorities of a subset of items along with the sequence access. Rather than a fixed priority for each item, we allow the priorities to change as keys get accessed. The setting has a wide range of applications in the real world. For instance, consider accessing data in a relational database. A sequence of access will likely access related items one after another. So even if the entries themselves get accessed with fixed frequencies, the distribution of the next item to be accessed can be highly dependent on the set of recently accessed items. Consider the access sequence

$$4, 2, 3, 4, 5, 2, 3, 4, 5, 2, 3, 4, 5, 1, 4$$

versus the access sequence

$$5, 2, 4, 2, 1, 4, 4, 5, 3, 3, 3, 4, 5, 4, 2.$$

In both trees, the item 4 is accessed the most frequently. So input dependent search trees should place it near the root. However, in the second sequence, the item 3 is accessed three consecutive times around the middle. An algorithm that's allowed to modify the tree dynamically can then modify the tree to place 3 closer to root during those calls. An illustration of this is in Figure 1.

Thus, it is natural for the item priorities to vary as more items get accessed. We attempt to adapt Treaps and B-Treaps to this setting, specifically one where the predictions can change as more items get accessed.

Both Treaps and B-Treaps can handle updates to priorities efficiently. In particular, updating the priority of an element from w_x to w'_x takes $O(|\log_2(w_x/w'_x)|)$ time for the Treap case and $O(|\log_B(w_x/w'_x)|)$ memory accesses for the B-Treap case. Denote $U(i)$ as the subset of items that update scores at time i and $w_{i,j}$ as the score for item j at time i . We show in Theorem 6.1 that the total cost for accessing the sequence $\mathbf{X}$ with the dynamic Treaps consists of two parts. The first part $O(n \log_B n + \sum_{i \in [m]} \log_B \frac{1}{w_{i,x(i)}})$ is the same as the static optimal bound and the second part $O(\sum_{i=2}^m \sum_{j \in U(i)} |\log_B \frac{w_{i,j}}{w_{i-1,j}}|)$ comes from updating scores. Hence, here is a trade-off between the costs of updating items and the benefits from the time-varying scores.

We study ways of designing composite priorities that cause this access cost to match known sequence-dependent access costs of binary trees (and their natural generalizations to B-Trees). Here, we focus on the working set bound, which says that the cost of accessing an item should be at most the logarithm of the number of distinct items until it gets accessed again. To obtain this

bound, we propose a new composite priority named *interval set priority*, based on the number of distinct elements between two occurrences of the same item around accessed at step i . We give the guarantees for the dynamic Treaps with the interval set priority in [Theorem 6.8](#). The dynamic B-Treaps further show the power of learning scores from data. While we have more data, we can quantify the dynamic environment in a more accurate way and thus improve the efficiency of the data structure.

Robustness to Prediction Inaccuracy Finally, we provide some preliminary work on the robustness of our data structures to inaccuracies in prediction oracles.

In the static case, we can directly relate the overhead of having inaccurate frequency predictions to the KL divergences between the estimates and the true frequencies. This is because our composite priority can take any estimate, so plugging in the estimates q_x gives that the overall access cost is exactly

$$m \cdot \sum_x q_x \log_2 \left(\frac{1}{p_x} \right),$$

which is exactly the cross entropy between $\mathbf{p}$ and $\mathbf{q}$. On the other hand, the KL divergence between $\mathbf{p}$ and $\mathbf{q}$ is exactly the cross entropy minus the entropy of $\mathbf{p}$. So we get that the overhead of building the tree using noisy estimators $\mathbf{q}$ instead of the true frequencies $\mathbf{p}$ is exactly m times the KL-divergence between $\mathbf{p}$ and $\mathbf{q}$. We formalize the argument above in [Section 4.3](#).

In the online/dynamic setting, we incorporate notions from the analyses of recommender systems: each prediction can be treated as a recommendation/score of the item for future access. Recommender systems are widely studied in statistics, machine learning, and data mining. A variety of metrics have been proposed for them, and we follow the survey by Zangerle and Bauer [\[ZB22\]](#), specifically the notion of mean-average-error. In [Theorem 6.9](#), we show that if the mean average error of the predicted priority vector is ϵ , our learned B-Trees handle an access sequence with an additional overhead of $m \log_B(1 + n\epsilon/m)$.

3 Related Work

Learned Index Models In recent years, there has been a surge of interest in integrating machine learning models into algorithm designs. A new field called *Algorithms with Predictions* [\[MV20\]](#) has garnered considerable attention, particularly in the use of machine learning models to predict input patterns to enhance performance. Examples of this approach include online graph algorithms with predictions [\[APT22\]](#), improved hashing-based methods such as Count-Min [\[CM05\]](#), and learning-augmented k -means clustering [\[EFSWZ21\]](#). Practical oracles for predicting desired properties, such as predicting item frequencies in a data stream, have been demonstrated empirically [\[HIKV19; JLLRW20\]](#).

Capitalizing on the existence of oracles that predict the properties of upcoming accesses, researchers are now developing more efficient learning-augmented data structures. Index structures in database management systems are one significant application of learning-augmented data structures. One key challenge in this domain is to create search algorithms and data structures that are efficient and adaptive to data whose nature changes over time. This has spurred interest in incorporating machine learning techniques to improve traditional search tree performance.

The first study on learned index structures [\[KBCDP18\]](#) used deep-learning models to predict the position or existence of records as an alternative to the traditional B-Tree or hash index. However, this study focused only on the static case. Subsequent research [\[FV20; DMYWDLZCGK+20; WZCWCX21\]](#) introduced dynamic learned index structures with provably efficient time and space

upper bounds for updates in the worst case. These structures outperformed traditional B-Trees in practice, but their theoretical guarantees were often trivial, with no clear connection between prediction quality and performance.

Beyond Worst-Case Analyses of Binary Trees Binary trees are among the most ubiquitous pointer-based data structures. While schemes without re-balancing do obtain $O(\log_2 n)$ time bounds in the average case, their behavior degenerates to $\Omega(n)$ on natural access sequences such as $1, 2, 3, \dots, n$. To remedy this, many tree balancing schemes with $O(\log_2 n)$ time worst-case guarantees have been proposed [AL63; GS78; CLRS09].

Creating binary trees optimal for their inputs has been studied since the 1970s. Given access frequencies, the static tree of optimal cost can be computed using dynamic programs or clever greedies [HT70; Meh75b; Yao82; KLR96]. However, the cost of such computations often exceeds the cost of invoking the tree. Therefore, a common goal is to obtain a tree whose cost is within a constant factor of the entropy of the data, multiple schemes do achieve this either on worst-case data [Meh75b], or when the input follows certain distributions [AM78].

A major disadvantage of static trees is that their cost on any permutation needs to be $\Omega(n \log_2 n)$. On the other hand, for the access sequence $1, 2, 3, \dots, n$, repeatedly bringing the next accessed element to the root gives a lower cost $O(n)$. This prompted Allen and Munro to propose the notion of self-organizing binary search trees. This scheme was extended to splay trees by Sleator and Tarjan [ST85b]. Splay trees have been shown to obtain many improved cost bounds based on temporal and spatial locality [ST85b; CMSS00; Col00; Iac05]. In fact, they have been conjectured to have access costs with a constant factor of optimal on any access sequence [Iac13]. Much progress has been made towards showing this over the past two decades [DHIKP09; DS09; CCS19; BCIKL20].

From the perspective of designing learning-augmented data structures, the dynamic optimality conjecture almost goes contrary to the idea of incorporating predictors. It can be viewed as saying that learned advice do not offer gains beyond constant factors, at least in the binary search tree setting. Nonetheless, the notion of access sequence, as well as access-sequence-dependent bounds, provides useful starting points for developing prediction-dependent search trees in online settings. In this paper, we choose to focus on bounds based on temporal locality, specifically, the working set bound. This is for two reasons: the spatial locality of an element’s next access is significantly harder to describe compared to the time until the next access; and the current literature on spatial locality-based bounds, such as dynamic finger tends to be much more involved [CMSS00; Col00]. We believe an interesting direction for extending our composite scores is to obtain analogs of the unified bound [Iac01; BCDI07] for B-Trees.

B-Trees and External Memory Model Parameterized B-Trees [BF03] have been studied to balance the runtime of read versus write operations, and several bounds have been shown with regard to the blocks of memory needed to be used during an operation. The optimality is discussed in both static and dynamic settings. Rosenberg and Snyder [RS81] compared the B-Tree with the minimum number of nodes (denoted as *compact*) with non-compact B-Trees and with time-optimal B-Trees. Bender et al. [BEHK16] considers keys have different sizes and gives a cache-oblivious static atomic-key B-Tree achieving the same asymptotic performance as the static B-Tree. When it comes to the dynamic setting, the trade-off between the cost of updates and accesses is widely studied [OCGO96; JNSSK97; JDO99; BGVW00; Yi12].

B-Treap were introduced by Golovin [Gol08; Gol09] as a way to give an efficient history-independent search tree in the external memory model. These studies revolved around obtaining $O(\log_B n)$ worst-case costs that naturally generalize Treaps. Specifically, for sufficiently small B

(as compared to n), Golovin showed a worst-case depth of $O(\frac{1}{\alpha} \log_B n)$ with high probability, where $\alpha > 0$ a parameter relating the limit on B to n . The running time of this structure has recently been improved by Safavi [SS23] via a two-layer design.

The large node sizes of B-Trees interact naturally with the external memory model, where memory is accessed in blocks of size B [BF03; Vit01]. The external memory model itself is widely used in data storage and retrieval [MA13], and has also been studied in conjunction with learned indices [FLV20]. There are a number of previous results discussing the trade-off between update and storage utilization [Bro14; Bro17; FHM19].

4 Statically Optimal Binary Trees via Learning

In this section, we show that the widely taught Treap data structure can, with small modifications, achieves the static optimality conditions typically sought after in previous studies of learned index structures [LLW22; HIKV19].

Definition 4.1 (Treap, [AS89]). *Let T be a Binary Search Tree over $[n]$ and $\text{priority} \in \mathbb{R}^n$ be a priority assignment on $[n]$. We say $(T, \text{priority})$ is a Treap if $\text{priority}_x \leq \text{priority}_y$ whenever x is a descendent of y in T .*

Given a priority assignment priority , one can construct a BST T such that $(T, \text{priority})$ is a Treap. T is built as follows: Take any $x^* \in \arg \max_x \text{priority}_x$ and build Treaps on $[1, x^* - 1]$ and $[x^* + 1, n]$ recursively using priority . Then, we just make x^* the parent of both Treaps. Notice that if priority_x 's are distinct, the resulting Treap is unique.

Observation 1. *Let $\text{priority} \in \mathbb{R}^n$, which assigns each item x to a unique priority. There is a unique BST T such that $(T, \text{priority})$ is a Treap.*

From now on, we always assume that priority has distinct values. Therefore, when priority is clear from the context, the term *Treap* is referred to the unique BST T . For each node $x \in [n]$, we use $\text{depth}(x)$ to denote its depth in T .

Given any two items $x, y \in [n]$, one can determine whether x is an ancestor of y in the Treap without traversing the tree. In particular, x is an ancestor of y if x 's priority is the largest among items $[x, y]$.

Observation 2. *Given any $x, y \in [n]$, x is an ancestor of y if and only if $\text{priority}_x = \max_{z \in [x, y]} \text{priority}_z$.*

A classical result of [AS89] states that if priorities are randomly assigned, the depth of the Treap cannot be too large.

Lemma 4.2 ([AS89]). *Let $U(0, 1)$ be the uniform distribution over the real interval $[0, 1]$. If $\text{priority} \sim U(0, 1)^n$, each Treap node x has depth $\Theta(\log_2 n)$ with high probability.*

Proof. Notice that $\text{depth}(x)$, the depth of item x in the Treap, is the number of ancestors of x in the Treap. Linearity of expectation yields

$$\begin{aligned} \mathbb{E}[\text{depth}(x)] &= \sum_{y \in [n]} \mathbb{E}[1 \text{ if } y \text{ is an ancestor of } x \text{ or } 0] \\ &= \sum_{y \in [n]} \Pr(y \text{ is an ancestor of } x) \\ &= \sum_{y \in [n]} \Pr\left(\text{priority}_y = \max_{z \in [x, y]} \text{priority}_z\right) = \sum_{y \in [n]} \frac{1}{|x - y + 1|} = \Theta(\log_2 n). \end{aligned}$$

□

Treaps can be made dynamic and support operations such as insertions and deletions.

Lemma 4.3 ([AS89]). *Given a Treap T and some item $x \in [n]$, x can be inserted to or deleted from T in $O(\text{depth}(x))$ -time.*

4.1 Learning-Augmented Treaps

In this section, we present the construction of composite priorities and prove the following theorem.

Theorem 4.4 (Learning-Augmented Treap via Composite Priorities). *Denote $\mathbf{w} = (w_1, \dots, w_n) \in \mathbb{R}^n$ as a score associated with each item in $[n]$ such that $\|\mathbf{w}\|_1 = O(1)$. Consider the following priority assignment of each item:*

$$\text{priority}_x \stackrel{\text{def}}{=} - \left\lfloor \log_2 \log_2 \frac{1}{w_x} \right\rfloor + \delta_x, \quad \delta_x \sim U(0, 1), \quad x \in [n]$$

where δ_x is drawn uniformly from $(0, 1)$ via a 11-wise independent hash function. The depth of any item $x \in [n]$ is $O(\log_2(1/w_x))$ in expectation.

Remark 4.5. *The data structure supports insertions and deletions naturally. Suppose the score of some node x changes from w to w' , the Treap can be maintained with $O(|\log_2(w'/w)|)$ rotations in expectation.*

For any item $x \in [n]$, we define x 's tier, τ_x , to be the integral part of its priority, i.e.,

$$\tau_x \stackrel{\text{def}}{=} \left\lfloor \log_2 \log_2 \frac{1}{w_x} \right\rfloor = -\lfloor \text{priority}_x \rfloor. \quad (1)$$

In addition, we define $S_t = \{x \in [n] \mid \tau_x = t\}$ for any t . For simplicity, we assume that $\tau_x \geq 0$ for any x WLOG. This is because $\tau_x < 0$ implies $w_x = \Omega(1)$ which can hold for only a constant number of items. We can always make them stay at the top of the Treap. This only increases the depths of others by a constant.

Let x start at any node and approach the root. The tier τ_x decreases. In other words, the smaller the τ_x , the smaller the depth $\text{depth}(x)$. However, there may be many items of the same tier τ_x . The ties are broken randomly due to the random offset $\delta_x \sim U(0, 1)$. Similar to the ordinary Treaps, where every item has the same tier, any item has $O(\log_2 |S_t|)$ ancestors with tier w in expectation. To prove the desired bound, we will show that $\log_2 |S_t| = O(2^t)$. Since each ancestor of item x has weight at most τ_x , the expected depth $\mathbb{E}[\text{depth}(x)]$ can be bound by $O(2^0 + 2^1 + \dots + 2^{\tau_x}) = O(2^{\tau_x}) = O(\log_2(1/w_x))$. First, let us bound the size of each S_w .

Lemma 4.6. *For any non-negative integer $t \geq 0$, $|S_t| = O(2^{O(2^t)})$.*

Proof. Observe that $x \in S_t$ if and only if

$$t \leq \log_2 \log_2(1/w_x) \leq t + 1, \text{ and } 2^{2^t} \leq \frac{1}{w_x} \leq 2^{2^{t+1}}.$$

However, there are only $O(\text{poly}(2^{2^{t+1}})) = O(2^{O(2^t)})$ such items because the total score $\|\mathbf{w}\|_1 = O(1)$. $\square$

Next, we bound the expected number of ancestors of item x in $S_w, w \leq w_x$.

Lemma 4.7. *Let $x \in [n]$ be any item and $t \leq \tau_x$ be a non-negative integer. The expected number of ancestors of x in S_t is at most $O(\log_2 |S_t|)$.*

Proof. First, we show that any $y \in S_t$ is an ancestor of x with probability no more than $1/|S_t \cap [x, y]|$. [Observation 2](#) says that y must have the largest priority among items $[x, y]$. Thus, a necessary condition for y being x 's ancestor is that y has the largest priority among items in $S_t \cap [x, y]$. However, priorities of items in $S_t \cap [x, y]$ are i.i.d. random variables of the form $-t + U(0, 1)$. Thus, the probability that priority_y is the largest among them is $1/|S_t \cap [x, y]|$.

Now, we can bound the expected number of ancestors of x in S_t as follows:

$$\begin{aligned} \mathbb{E}[\text{number of ancestors of } x \text{ in } S_t] &= \sum_{y \in S_t} \Pr(y \text{ is an ancestor of } x) \\ &\leq \sum_{y \in S_t} \frac{1}{|S_t \cap [x, y]|} \\ &\leq 2 \cdot \sum_{u=1}^{|S_t|} \frac{1}{u} = O(\log_2 |S_t|), \end{aligned}$$

where the second inequality comes from the fact that for a fixed value of u , there are at most two items $y \in S_t$ with $|S_t \cap [x, y]| = u$ (one with $y \leq x$, the other with $y > x$). $\square$

Now we are ready to prove [Theorem 4.4](#).

Proof of Theorem 4.4. Let $x \in [n]$ be any item. Linearity of expectation yields

$$\begin{aligned} \mathbb{E}[\text{depth}(x)] &= \sum_{t=0}^{\tau_x} \mathbb{E}[\text{number of ancestors of } x \text{ in } S_t] \\ &\stackrel{\text{Lemma 4.7}}{\lesssim} \sum_{t=0}^{\tau_x} \log_2 |S_t| \\ &\stackrel{\text{Lemma 4.6}}{\lesssim} \sum_{t=0}^{\tau_x} 2^t \lesssim 2^{\tau_x}. \end{aligned}$$

We conclude the proof by observing that

$$\tau_x \leq \log_2 \log_2 \frac{1}{w_x} \leq \tau_x + 1, \text{ and } 2^{\tau_x} \leq \log_2 \frac{1}{w_x}.$$

$\square$

4.2 Static Optimality

We present a priority assignment for constructing statically optimal Treaps given item frequencies. Given any access sequence $\mathbf{X} = (x(1), \dots, x(m))$, we define f_x for any item x , to be its *frequency* in $\mathbf{X}$, i.e. $f_x := |\{i \in [m] \mid x(i) = x\}|$, $x \in [n]$. For simplicity, we assume that every item is accessed at least once, i.e., $f_x \geq 1$, $x \in [n]$. We prove the following technical result which is a simple application of [Theorem 4.4](#):

Theorem 4.8 (Static Optimality). *For any item $x \in [n]$, we set its priority as*

$$\text{priority}_x := - \left\lfloor \log_2 \log_2 \frac{m}{f_x} \right\rfloor + \delta_x, \delta_x \sim U(0, 1).$$

In the corresponding Treap, each node x has expected depth $O(\log_2(m/f_x))$. Therefore, the total time for processing the access sequence is $O(\sum_x f_x \log_2(m/f_x))$, which matches the performance of the optimal static BSTs up to a constant factor.

Proof. Given item frequencies $\mathbf{f}$, we define the following $\mathbf{w}$ assignment:

$$w_x := \frac{f_x}{m}, \quad x \in [n]. \quad (2)$$

One can verify that $\|\mathbf{w}\|_1 = O(1)$, therefore [Theorem 4.4](#) yields that the expected depth of each item x is $O(\log_2(m/f_x))$. $\square$

4.3 Robustness Guarantees

In practice, one could only estimate $q_x \approx p_x = f_x/m, x \in [n]$. A natural question arises: how does the estimation error affect the performance? In this section, we analyze the drawback in performance given estimation errors. As a result, we will show that our Learning-Augmented Treaps are robust against noise and errors.

For each item $x \in [n]$, define $p_x = f_x/m$ to be the relative frequency of item x . One can view $\mathbf{p}$ as a probability distribution over $[n]$. Using the notion of entropy, one can express [Theorem 4.8](#) as the following claim:

Definition 4.9 (Entropy). *Given a probability distribution $\mathbf{p}$ over $[n]$, define its Entropy as $\text{Ent}(\mathbf{p}) := \sum_x p_x \log_2(1/p_x) = \mathbb{E}_{x \sim \mathbf{p}}[\log_2(1/p_x)]$.*

Corollary 4.10. *In [Theorem 4.8](#), the expected depth of each item x is $O(\log_2(1/p_x))$ and the expected total cost is $O(m \cdot \text{Ent}(\mathbf{p}))$, where $\text{Ent}(\mathbf{p}) = \sum_x p_x \log_2(1/p_x)$ measures the entropy of the distribution $\mathbf{p}$.*

The appearance of entropy in the runtime bound suggests that some more related notations would appear in the analysis. Let us present several related notions.

Definition 4.11 (Cross Entropy). *Given two distributions $\mathbf{p}, \mathbf{q}$ over $[n]$, define its Cross Entropy as $\text{Ent}(\mathbf{p}, \mathbf{q}) := \sum_x p_x \log_2(1/q_x) = \mathbb{E}_{x \sim \mathbf{p}}[\log_2(1/q_x)]$.*

Definition 4.12 (KL Divergence). *Given two distributions $\mathbf{p}, \mathbf{q}$ over $[n]$, define its KL Divergence as $\text{D}_{\text{KL}}(\mathbf{p}, \mathbf{q}) = \text{Ent}(\mathbf{p}, \mathbf{q}) - \text{Ent}(\mathbf{p}) = \sum_x p_x \log_2(p_x/q_x)$.*

First, we analyze the run time given frequency estimations $\mathbf{q}$.

Theorem 4.13. *Given an estimation $\mathbf{q}$ on the relative frequencies $\mathbf{p}$. For any item $x \in [n]$, we draw a random number $\delta_x \sim U(0, 1)$ and set its priority as*

$$\text{priority}_x := - \left\lceil \log_2 \log_2 \frac{1}{q_x} \right\rceil + \delta_x.$$

In the corresponding Treap, each node x has expected depth $O(\log_2(1/q_x))$. Therefore, the total time for processing the access sequence is $O(m \cdot \text{Ent}(\mathbf{p}, \mathbf{q}))$.

Proof. Define score $s(x) = 1/q_x$ for each item $x \in [n]$. Clearly, s is smooth and we can apply [Theorem 4.4](#) to prove the bound on the expected depths. The total time for processing the access sequence is, by definition,

$$O \left(\sum_{x \in [n]} f_x \log_2 \frac{1}{q_x} \right) = O \left(m \cdot \sum_{x \in [n]} p_x \log_2 \frac{1}{q_x} \right) = O(m \cdot \text{Ent}(\mathbf{p}, \mathbf{q})).$$

$\square$

Using the theorem, one can relate the drawback in the performance given $\mathbf{q}$ in terms of KL Divergence.

Corollary 4.14. *In the setting of [Theorem 4.13](#), the extra time spent compared to using $\mathbf{p}$ to build the Treap is $O(m \cdot \text{D}_{\text{KL}}(\mathbf{p}, \mathbf{q}))$ in expectation.*

4.4 Analysis of Other Variations

We discuss two different priority assignments. For each assignment, we construct an input distribution that creates a larger expected depth than [Theorem 4.8](#). We define the distribution $\mathbf{p}$ as $p_x = f_x/m, x \in [n]$.

The first priority assignment is used in [\[LLW22\]](#). They assign priorities according to p_x entirely, i.e., $\text{priority}_x = p_x, x \in [n]$. Assuming that items are ordered randomly, and $\mathbf{p}$ is a Zipfian distribution, [\[LLW22\]](#) shows *Static Optimality*. However, it does not generally hold, and the expected access cost could be $\Omega(n)$.

Theorem 4.15. *Consider the priority assignment that assigns the priority of each item to be $\text{priority}_x := p_x, x \in [n]$. There is a distribution $\mathbf{p}$ over $[n]$ such that the expected access time, $\mathbb{E}_{x \sim \mathbf{p}}[\text{depth}(x)] = \Omega(n)$.*

Proof. We define for each item x , $p_x := \frac{2(n-x+1)}{n(n+1)}$. One could easily verify that $\mathbf{p}$ is a distribution over $[n]$. In addition, the smaller the item x , the larger the priority priority_x . Thus, by the definition of Treaps, item x has depth x . The expected access time of x sampled from $\mathbf{p}$ can be lower bounded as follows:

$$\begin{aligned} \mathbb{E}_{x \sim \mathbf{p}}[\text{depth}(x)] &= \sum_{x \in [n]} p_x \cdot \text{depth}(x) \\ &= \sum_{x \in [n]} \frac{2(n-x+1)}{n(n+1)} \cdot x \\ &= \frac{2}{n(n+1)} \sum_{x \in [n]} x(n-x+1) \\ &\gtrsim \frac{2}{n(n+1)} \cdot n^3 \gtrsim n. \end{aligned}$$

□

Next, we consider a very similar assignment to ours.

Theorem 4.16. *Consider the following priority assignment that sets the priority of each node x as $\text{priority}_x := -\lfloor \log_2 1/p_x \rfloor + \delta_x, \delta_x \sim U(0, 1)$. There is a distribution $\mathbf{p}$ over $[n]$ such that the expected access time, $\mathbb{E}_{x \sim \mathbf{p}}[\text{depth}(x)] = \Omega(\log_2^2 n)$.*

Proof. We assume WLOG that n is an even power of 2. Define $K = \frac{1}{2} \log_2 n$. We partition $[n]$ into $K+1$ segments $S_1, \dots, S_K, S_{K+1} \subseteq [n]$. For $i = 1, 2, \dots, K$, we add $2^{1-i} \cdot n/K$ elements to S_i . Thus, S_1 has n/K elements, S_2 has $n/2K$, and S_K has $\sqrt{n}/K$ elements. The rest are moved to S_{K+1} .

Now, we can define the distribution $\mathbf{p}$. Elements in S_{K+1} have zero-mass. For $i = 1, 2, \dots, K$, elements in S_i has probability mass $2^{i-1}/n$. One can directly verify that $\mathbf{p}$ is indeed a probability distribution over $[n]$.

In the Treap with the given priority assignment, S_i forms a subtree of expected height $\Omega(\log_2 n)$ since $|S_i| \geq n^{1/3}$ for any $i = 1, 2, \dots, K$ ([Lemma 4.2](#)). In addition, every element of S_i passes through $S_{i+1}, S_{i+2}, \dots, S_K$ on its way to the root since they have strictly larger priorities. Therefore, the expected depth of element $x \in S_i$ is $\Omega((K-i) \log_2 n)$. One can lower bound the expected access

time (which is the expected depth) as:

$$\begin{aligned}
\mathbb{E}_{x \sim \mathbf{p}}[\text{depth}(x)] &\gtrsim \sum_{i=1}^K \sum_{x \in S_i} p_x \cdot (K - i) \cdot \log_2 n \\
&= \sum_{i=1}^K \mathbf{p}(S_i) \cdot (K - i) \cdot \log_2 n \\
&= \sum_{i=1}^K \frac{1}{K} \cdot (K - i) \cdot \log_2 n \gtrsim K \log_2 n \gtrsim \log_2^2 n,
\end{aligned}$$

where we use $\mathbf{p}(S_i) = |S_i| \cdot 2^{i-1}/n = 1/K$ and $K = \Theta(\log_2 n)$. That is, the expected access time is at least $\Omega(\log_2^2 n)$. $\square$

5 Learning-Augmented B-Trees: Static Optimality

We now extend the ideas above, specifically the composite priority notions, to B-Trees in the *External Memory Model*, and also obtain static optimality in this model. This model is also the core of our analyses in the online settings in the next section (Section 6).

We first formalize this extension by incorporating our composite priorities with the B-Treap data structure from [Gol09] and introducing offsets in priorities.

5.1 Learning-Augmented B-Treaps

Lemma 5.1 (B-Treap, [Gol09]). *Given the unique binary Treap $(T, \text{priority}_x)$ over the set of items $[n]$ with their associated priorities, and a target branching factor $B = \Omega(\ln^{1/(1-\alpha)} n)$ for some $\alpha > 0$. Assuming priority_x are drawn uniformly from $(0, 1)$ using an 11-wise independent hash function, we can maintain a B-Tree T^B , called the B-Treap, uniquely defined by T . This allows operations such as Lookup, Insert, and Delete of an item to touch $O(\frac{1}{\alpha} \log_B n)$ nodes in T^B in expectation.*

In particular, if $B = O(n^{1/2-\delta})$ for some $\delta > 0$, all above performance guarantees hold with high probability.

The main technical theorem is the following:

Theorem 5.2 (Learning-Augmented B-Treap via Composite Priorities). *Denote $\mathbf{w} = (w_1, \dots, w_n) \in \mathbb{R}^n$ as a score associated with each element of $[n]$ such that $\|\mathbf{w}\|_1 = O(1)$ and a branching factor $B = \Omega(\ln^{1/(1-\alpha)} n)$, there is a randomized data structure that maintains a B-Tree T^B over U such that*

1. *Each item x has expected depth $O(\frac{1}{\alpha} \log_B(1/w_x))$.*
2. *Insertion or deletion of item x into/from T touches $O(\frac{1}{\alpha} \log_B(1/w_x))$ nodes in T^B in expectation.*
3. *Updating the weight of item x from w to w' touches $O(\frac{1}{\alpha} |\log_B(w'/w)|)$ nodes in T^B in expectation.*

We consider the following priority assignment scheme: For any x and its corresponding score w_x , we always maintain:

$$\text{priority}_x := -\lfloor \log_4 \log_B \frac{1}{w_x} \rfloor + \delta, \quad \delta \sim U(0, 1).$$

In addition, if $B = O(n^{1/2-\delta})$ for some $\delta > 0$, all above performance guarantees hold with high probability $1 - \delta$.

The learning-augmented B-Treap is created by applying [Lemma 5.1](#) to a partition of the binary Treap T . Each item x has a priority in the binary Treap T , defined as:

$$\text{priority}_x = - \left\lfloor \log_4 \log_B \frac{1}{w_x} \right\rfloor + \delta_x, \delta_x \sim U(0, 1), \text{ for all } x \in U \quad (3)$$

We then partition the binary Treap T based on each item's tier. The tier of an item is defined as the absolute value of the integral part of its priority, i.e., $\tau_x \stackrel{\text{def}}{=} \lfloor \log_4 \log_B(1/w_x) \rfloor$.

Proof of [Theorem 5.2](#). To formally construct and maintain T^B , we follow these steps:

1. Start with a binary Treap $(T, \text{priority}_x)$ with priorities defined using equation (3).
2. Decompose T into sub-trees based on each item's tier, resulting in a set of maximal sub-trees with items sharing the same tier.
3. For each T_i , apply [Lemma 5.1](#) to maintain a B-Treap T_i^B .
4. Combine all the B-Treaps into a single B-Tree, such that the parent of $\text{root}(T_i^B)$ is the B-Tree node containing the parent of $\text{root}(T_i)$.

Now, let's analyze the depth of each item x . Keep in mind that any item y in the same B-Tree node shares the same tier. Therefore, we can define the tier of each B-Tree node as the tier of its items.

Suppose $x_1, x_2, \dots$ are the B-Tree nodes we encounter until we reach x . The tiers of these nodes are in non-increasing order, that is, $\tau_{x_i} \geq \tau_{x_{i+1}}$ for any i . We'll define C_t as the number of items of tier t for any t . As per the definition (refer to equation (3)), we have:

$$C_t = O(B^{4^t}), \text{ for all } t$$

Using [Lemma 5.1](#) and the fact that $B = O(C_t^{1/4}), t \geq 1$, we find that the number of nodes among x_i of tier t is $O(\frac{1}{\alpha} \log_B C_t) = O(4^t/\alpha)$ with high probability. As a result, the number of nodes touched until reaching x is, with high probability:

$$\sum_{t=0}^{\tau_x} O(4^t/\alpha) = O(4^{\tau_x}/\alpha) = O\left(\frac{1}{\alpha} \log_B \frac{1}{w_x}\right)$$

This analysis is also applicable when performing Lookup, Insert, and Delete operations on item x .

The number of nodes touched when updating an item's weight can be derived from first deleting and then inserting the item.

□

We also show that this mechanism is robust to noisy predictions of item frequencies. Let $x(1), x(2), \dots, x(m)$ represent an access sequence of length m . We define the density of each item x as follows:

$$p_x \stackrel{\text{def}}{=} \frac{|\{i \in [m] \mid x(i) = x\}|}{m}$$

Note that $\mathbf{p} \in \mathbb{R}_+^n$ defines a probability distribution over all items. A static B-Tree is statically optimal if the depth of each item x is:

$$\text{depth}(x) = O\left(\log_B \frac{1}{p_x}\right), \text{ for all } x \in [n]$$

5.2 Static Optimality

If we are given the density $\mathbf{p}$, we can achieve *Static Optimality* using [Theorem 5.2](#) with weights $w_x \stackrel{\text{def}}{=} p_x$.

Lemma 5.3 (Static Optimality for B-Treaps). *Given the density $\mathbf{p}$ of each item over the access sequence $x(1), x(2), \dots, x(m)$, and a branching factor $B = \Omega(\ln^{1.1} n)$, there exists a randomized data structure that maintains a B-Tree T^B over $[n]$ such that each item x has an expected depth of $O(\log_B 1/p_x)$. That is, T^B achieves Static Optimality (SO), meaning the total number of nodes touched is $O(\text{OPT}_B^{\text{static}})$ in expectation, where:*

$$\text{OPT}_B^{\text{static}} \stackrel{\text{def}}{=} m \cdot \sum_{x \in [n]} p_x \log_B \frac{1}{p_x} \quad (4)$$

Furthermore, if $B = O(n^{1/2-\delta})$ for some $\delta > 0$, all above performance guarantees hold with high probability.

In practice, we would not have access to the exact density $\mathbf{p}$ but an inaccurate prediction $\tilde{\mathbf{p}} \approx \mathbf{p}$. We assume WLOG that $|p(x)|, |\tilde{p}(x)| \geq \Theta(1/n)$. Nevertheless, we will show that B-Treap performance is robust to the error. Specifically, we analyze the performance under various notions of error in the prediction. The notions listed here are the ones used for learning discrete distributions (refer to [\[Can20\]](#) for a comprehensive discussion).

Corollary 5.4 (Kullback—Leibler (KL) Divergence). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $d_{KL}(\mathbf{p}; \tilde{\mathbf{p}}) = \sum_x p_x \ln(p_x/\tilde{p}_x) \leq \varepsilon$, the total number of touched nodes is*

$$O\left(\text{OPT}_B^{\text{static}} + \frac{\varepsilon m}{\ln B}\right)$$

Proof. Given the inaccurate prediction $\tilde{\mathbf{p}}$, the total number of touched nodes in T^B is

$$O\left(\sum_x m \cdot p_x \log_B \frac{1}{\tilde{p}_x}\right) = O\left(\sum_x m \cdot p_x \log_B \frac{1}{p_x} + m \cdot \sum_x p_x \log_B \frac{p_x}{\tilde{p}_x}\right) = O(\text{OPT}_B^{\text{static}} + m \frac{d_{KL}(\mathbf{p}; \tilde{\mathbf{p}})}{\ln B})$$

□

Corollary 5.5 (χ^2). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $\chi^2(\mathbf{p}; \tilde{\mathbf{p}}) = \sum_x (p_x - \tilde{p}_x)^2 / \tilde{p}_x \leq \varepsilon$, the total number of touched nodes is*

$$O\left(\text{OPT}_B^{\text{static}} + \frac{\varepsilon m}{\ln B}\right)$$

Proof. The corollary follows from [Corollary 5.5](#) and the fact $d_{KL}(\mathbf{p}; \tilde{\mathbf{p}}) \leq \chi^2(\mathbf{p}; \tilde{\mathbf{p}}) \leq \varepsilon$. $\square$

Corollary 5.6 (L_∞ Distance). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $\|\mathbf{p} - \tilde{\mathbf{p}}\|_\infty \leq \varepsilon$, the total number of touched nodes is*

$$O\left(OPT_B^{static} + m \log_B(1 + \varepsilon n)\right)$$

Proof. For item x with its marginal probability smaller than $1/1000n$, its expected depth in the B-Treap is $O(\log_B n)$ using either p_x or $\tilde{p}_x$ as its score. If item x 's marginal probability is at least $1/1000n$, the L_∞ distance implies that

$$\frac{p_x}{\tilde{p}_x} = 1 + \frac{p_x - \tilde{p}_x}{\tilde{p}_x} \leq 1 + \frac{\varepsilon}{1/(1000n)} = 1 + 1000(1 + \varepsilon n)$$

Therefore, item x 's expected depth in the B-Treap with score $\tilde{\mathbf{p}}$ is roughly

$$O\left(\log_B \frac{1}{p_x} + \log_B \frac{p_x}{\tilde{p}_x}\right) \leq O\left(\log_B \frac{1}{p_x} + \log_B(1 + \varepsilon n)\right)$$

The corollary follows. $\square$

Corollary 5.7 (L_2 Distance). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $\|\mathbf{p} - \tilde{\mathbf{p}}\|_2 \leq \varepsilon$, the total number of touched nodes is*

$$O\left(OPT_B^{static} + m \log_B(1 + \varepsilon n)\right)$$

Proof. This claim follows from [Corollary 5.6](#) and the fact $\|\mathbf{p} - \tilde{\mathbf{p}}\|_\infty \leq \|\mathbf{p} - \tilde{\mathbf{p}}\|_2 \leq \varepsilon$. $\square$

Corollary 5.8 (Total Variation). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $d_{TV}(\mathbf{p}, \tilde{\mathbf{p}}) = 0.5\|\mathbf{p} - \tilde{\mathbf{p}}\|_1 \leq \varepsilon$, the total number of touched nodes is*

$$O\left(OPT_B^{static} + m \log_B(1 + \varepsilon n)\right)$$

Proof. This claim follows from [Corollary 5.6](#) and the fact $\|\mathbf{p} - \tilde{\mathbf{p}}\|_\infty \leq \|\mathbf{p} - \tilde{\mathbf{p}}\|_1 \leq 2\varepsilon$. $\square$

Corollary 5.9 (Hellinger Distance). *If we are given a density prediction $\tilde{\mathbf{p}}$ such that $d_H(\mathbf{p}, \tilde{\mathbf{p}}) = 0.5\|\sqrt{\mathbf{p}} - \sqrt{\tilde{\mathbf{p}}}\|_2 \leq \varepsilon$, the total number of touched nodes is*

$$O\left(OPT_B^{static} + m \log_B(1 + \varepsilon n)\right)$$

Proof. This claim follows from [Corollary 5.6](#) and the fact $\|\mathbf{p} - \tilde{\mathbf{p}}\|_\infty \leq 2\sqrt{2}d_H(\mathbf{p}, \tilde{\mathbf{p}}) \leq 2\sqrt{2}\varepsilon$. $\square$

6 Dynamic Learning-Augmented B-Trees

In this section, we first construct the dynamic B-Treaps and give the guarantees when we have access to the real-time priorities for each item. Then we show that our Treaps are robust to the estimation error of the time-varying priorities. Finally, we consider a specific score that is highly related to the working-set theorem [\[ST85a\]](#).

6.1 General Results for Dynamic B-Trees

Dynamic B-Trees Setting. Given n items, denoted as $[n] = \{1, \dots, n\}$, and a sequence of access sequence $\mathbf{X} = (x(1), \dots, x(m))$, where $x(i) \in [n]$. At time $i \in [m]$, there exists some time-dependent priority w_{ij} for each item $j, j \in [n]$. Denote $\mathbf{w}(i) = (w_{i,1}, \dots, w_{i,n}) \in \mathbb{R}^n$ as the time-varying priority vector, and $\tilde{\mathbf{w}}(i) = (\tilde{w}_{i,1}, \dots, \tilde{w}_{i,n})$ as its estimator. We aim to maintain a data structure that achieves a small total cost for accessing the sequence S given inaccurate score estimators $\tilde{\mathbf{w}}_i, i \in [m]$.

We first show the guarantees when we are given the accurate time-varying priorities $\mathbf{w}(i), i \in [m]$ in [Theorem 6.1](#). The proof is an application of [Theorem 5.2](#), where the priority function dynamically changes as time goes on, rather than the *Static Optimality* case where the priority is fixed beforehand. Here for any vector $\mathbf{w}$, we write $\log \mathbf{w}$ as the vector taking the element-wise log on $\mathbf{w}$.

Theorem 6.1 (Dynamic B-Treap with Given Priorities). *Given the time-varying priority vector $\mathbf{w}(i) \in \mathbb{R}^n, i \in [m]$ satisfying $\|\mathbf{w}(i)\|_1 = O(1)$ and a branching factor $B = \Omega(\ln^{1.1} n)$, there is a randomized data structure that maintains a B-Tree T^B over $[n]$ such that when accessing the item $x(i)$ at time i , the expected depth of item $x(i)$ is $O(\log_B \frac{1}{w_{i,x(i)}})$. The expected total cost for processing the whole access sequence $\mathbf{X}$ is*

$$\text{cost}(\mathbf{X}, \mathbf{w}) = O\left(n \log_B n + \sum_{i=1}^m \log_B \frac{1}{w_{i,x(i)}} + \sum_{i=2}^m \|\log_B \mathbf{w}(i) - \log_B \mathbf{w}(i-1)\|_1\right).$$

Furthermore, for $\delta > 0$, by setting $B = O(n^{1/2-\delta})$, the guarantees hold with probability $1 - \delta$.

Proof. Initially, we set the priority for all items to be 1, and insert all items into the Treap. For any time $i \in [n]$, for $j \in [n]$ such that $w_{i-1,j} \neq w_{i,j}$, we set

$$\text{priority}_j^{(i)} := -\lfloor \log_4 \log_B \frac{1}{w_{i,j}} \rfloor + \delta_{ij}, \delta_{ij} \sim U(0, 1).$$

Since $\|\mathbf{w}(i)\|_1 = O(1), i \in [m]$, by [Theorem 5.2](#), the expected depth of item $s(i)$ is $O(\log_B \frac{1}{w_{i,x(i)}})$. The total cost for processing the sequence consists of both accessing $x(i)$ and updating the priorities. The expected total cost for all the accesses is

$$O\left(\sum_{i=1}^m \log_B \frac{1}{w_{i,x(i)}}\right).$$

Then we will calculate the cost to update the Treap. Updating the priority of j from $w_{i-1,j}$ to $w_{i,j}$ has cost $O(|\log_B(w_{i-1,j}/w_{i,j})|)$. Hence we can bound the expected total cost for maintaining the Treap by

$$O\left(n \log_B n + \sum_{i=2}^m \sum_{j=1}^n \left| \log_B \frac{w_{i-1,j}}{w_{i,j}} \right| \right).$$

Together the expected total cost is

$$O\left(n \log_B n + \sum_{i=1}^m \log_B \frac{1}{w_{i,x(i)}} + \sum_{i=2}^m \|\log_B \mathbf{w}(i) - \log_B \mathbf{w}(i-1)\|_1\right).$$

The high probability bound follows similarly as [Theorem 5.2](#). □

Next, we will give the guarantees for the dynamic B-Treaps with predicted priorities learned by a machine learning oracle.

Theorem 6.2 (Dynamic B-Treap with Predicted Scores). *Given the time-varying priority vector $\mathbf{w}(i) \in \mathbb{R}^n, i \in [m]$ and its estimator $\tilde{\mathbf{w}}(i) \in \mathbb{R}_+^n$ satisfying $\|\tilde{\mathbf{w}}(i)\|_1 = O(1)$, $\tilde{w}_{i,j} \geq 1/\text{poly}(n)$. Denote $\varepsilon_{i,j} = \tilde{w}_{i,j} - w_{i,j}$. Then for a branching factor $B = \Omega(\ln^{1.1} n)$, there is a randomized data structure that maintains a B-Tree T^B over $[n]$ such that when accessing $x_{s(i)}$ at time i , the expected depth of item $x(i)$ is $O\left(\log_B\left(\frac{1}{w_{i,x(i)}}\left(1 + n|\varepsilon_{i,x(i)}|\right)\right)\right)$. The expected total cost for processing the whole access sequence $\mathbf{X}$ given the predicted priority $\tilde{\mathbf{w}}$ is*

$$\text{cost}(\mathbf{X}, \tilde{\mathbf{w}}) = O\left(n \log_B n + \sum_{i=1}^m \log_B\left(\frac{1}{w_{i,x(i)}}\left(1 + n|\varepsilon_{i,x(i)}|\right)\right) + \sum_{i=2}^m \|\log_B \tilde{\mathbf{w}}(i) - \log_B \tilde{\mathbf{w}}(i-1)\|_1\right).$$

Furthermore, for $\delta > 0$, by setting $B = O(n^{1/2-\delta})$, the guarantees hold with probability $1 - \delta$.

Proof. For any $i \in [m], j \in [n]$, we have the following bounds.

$$O\left(\log_B \frac{1}{\tilde{w}_{i,j}}\right) = O\left(\log_B \frac{1}{w_{i,j}} + \log_B\left(1 - \frac{\varepsilon_{i,j}}{\tilde{w}_{i,j}}\right)\right) \leq O\left(\log_B \frac{1}{w_{i,j}} + \log_B(1 + |\varepsilon_{i,j}|n)\right)$$

Then we apply [Theorem 6.1](#) with priority $\tilde{\mathbf{w}}$, and get the expected depth of $x(i)$ is

$$O\left(\log_B \frac{1}{\tilde{w}_{i,j}}\right) = O\left(\log_B \frac{1}{w_{i,x(i)}} + \log_B(1 + |\varepsilon_{i,x(i)}|n)\right)$$

The expected total cost is

$$\text{cost}(\mathbf{X}, \tilde{\mathbf{w}}) = O\left(n \log_B n + \sum_{i=1}^m \log_B\left(\frac{1}{w_{i,x(i)}}\left(1 + n|\varepsilon_{i,x(i)}|\right)\right) + \sum_{i=2}^m \|\log_B \tilde{\mathbf{w}}(i) - \log_B \tilde{\mathbf{w}}(i-1)\|_1\right)$$

□

6.2 Interval Set Priority

It is worth noting that the total cost depends on the items that change priorities at each time. If the priorities for all items keep changing throughout accessing, $\log_B \tilde{\mathbf{w}}(i) - \log_B \tilde{\mathbf{w}}(i-1)$ is a dense vector, thus leading to a high cost to update priorities. So an ideal time-varying priority should only change the scores of a constant number of items at each time. Denote $U(i) = \{w_{i-1,j} \neq w_{i,j}, j \in [n]\}$ as the set of items that change priorities at time i . We define a priority, named *interval priority*, based on the number of distinct elements between two neighbor occurrences of the same item. We can show that at each time i , $|U(i)| = 1$.

Given an access sequence $\mathbf{X} = (x(1), \dots, x(m))$, at any time $i \in [m]$, we first define the previous and next access of item j in [Definition 6.3](#), and then define the interval-set size as the number of distinct items between previous and next access of j in [Definition 6.4](#). We also define the working-set size as the number of distinct items between the previous access of j to time i in [Definition 6.5](#). Finally, we define the interval set priority in [Definition 6.6](#). See [Figure 2](#) for the illustration.

Definition 6.3 (Previous and Next Access $\text{prev}(i, x)$ and $\text{next}(i, x)$). *Let $\text{prev}(i, x)$ be the previous access of item x at or before time i , i.e., $\text{prev}(i, x) := \max\{i' \leq i \mid x(i') = x\}$. Let $\text{next}(i, x)$ to be the next access of item x after time i , i.e., $\text{next}(i, x) := \min\{i' > i \mid x(i') = x\}$.*

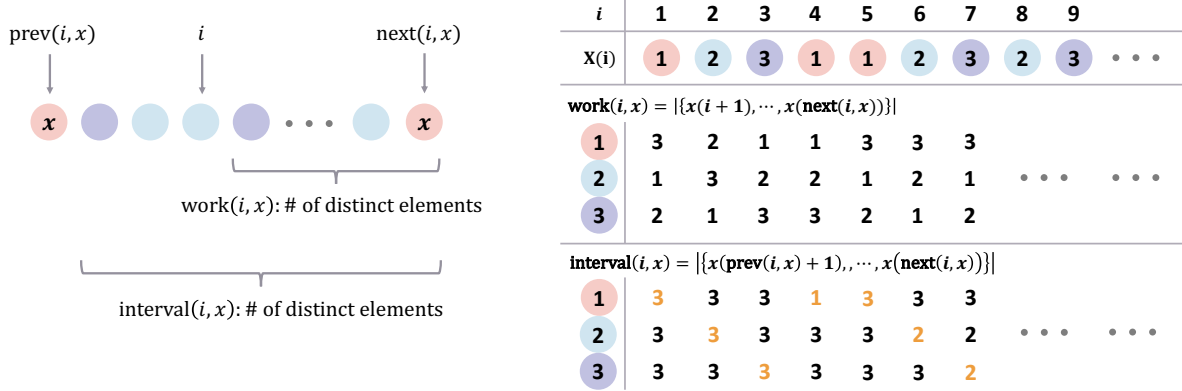


Figure 2: An example of $n = 3$, $\mathbf{X} = (1, 2, 3, 1, 1, 2, 3, \dots)$. For any i, x , $\text{work}(i)$ is a permutation of $[n]$; $\text{interval}(i, x) \geq \text{work}(i, x)$; $\text{interval}(i, x)$ changes only when $x(i) = x$ (highlighted in orange).

Definition 6.4 (Interval-Set Size $\text{interval}(i, x)$). Define the Interval-Set Size $\text{interval}(i, x)$ to be the number of distinct items accessed between the previous access of item x at or before time i and the next access of item x after time i . That is,

$$\text{interval}(i, x) := |\{x(\text{prev}(i, x) + 1), \dots, x(\text{next}(i, x))\}|.$$

If x does not appear after time i , we define $\text{interval}(i, x) := n$.

Definition 6.5 (Working-Set Size $\text{work}(i, x)$). Define the Working-Set Size $\text{work}(i, x)$ to be the number of distinct items accessed between time i and the next access of item x after time i . That is,

$$\text{work}(i, x) := |\{x(i + 1), \dots, x(\text{next}(i, x))\}|.$$

If x does not appear after time i , we define $\text{work}(i, x) := n$.

Definition 6.6 (Interval Set Priority $\text{isp}(i, x)$). Define the time-varying priority as the reciprocal of the square of one plus interval-set size. That is,

$$\text{isp}(i, x) = \frac{1}{(1 + \text{interval}(i, x))^2}$$

Next, we will show that the interval set priority is $O(1)$ for any time $i \in [m]$ in [Lemma 6.7](#). The proof has three steps. Firstly, the working-set size at time i is always a permutation of $[n]$. Secondly, for any $i \in [m], x \in [n]$, the interval-set size is always no less than the working-set size. Therefore, for any $i \in [m]$, the l_1 norm of interval set priority vector $\text{isp}(i) = (\text{isp}(i, 1), \dots, \text{isp}(i, n))$ can be upper bounded by $\sum_{j=1}^n 1/(1 + j)^2 = O(1)$.

Lemma 6.7 (Norm Bound for Interval Set Priority). Fix any timestamp $i \in [m]$,

$$\sum_{j=1}^n \text{isp}(i, j) = O(1).$$

Proof. We first show that at any time $i \in [m]$, the working-set size is a permutation of $[n]$. By definition, $\text{work}(i, x)$ is the number of items y such that $\text{next}(i, y) \leq \text{next}(i, x)$. Let $\pi_1, \dots, \pi_n$ be a

permutation of all items $[n]$ in the order of increasing $\text{next}(i, x)$. Then for any item x , $\text{work}(i, x)$ is the index of x in π . So the sum of the reciprocal of squared $\text{work}(i, x)$ is upper bounded by

$$\sum_{j=1}^n \frac{1}{(1 + \text{work}(i, j))^2} = \sum_{j=1}^n \frac{1}{(1 + j)^2} = \Theta(1).$$

Secondly, recall that $\text{prev}(i, x) \leq i$, and hence for any $i \in [m], x \in [n]$, $\text{interval}(i, x) \geq \text{work}(i, x)$. So we have the upper bound for interval set priority as follows.

$$\sum_{j=1}^n \text{isp}(i, j) = \sum_{j=1}^n \frac{1}{(1 + \text{interval}(i, x))^2} \leq \sum_{j=1}^n \frac{1}{(1 + \text{work}(i, j))^2} = O(1).$$

□

Since we have shown the interval set priority has constant l_1 norm and in each timestamp, we only update one item's priority. We are ready to prove and show the efficiency of the corresponding B-Treap.

Theorem 6.8 (Dynamic B-Treaps with Interval Set Priority). *With known interval set priority $\text{isp}(i, x(i)), i \in [m]$ and a branching factor $B = \Omega(\ln^{1.1} n)$, there is a randomized data structure that maintains a B-Tree T^B over $[n]$ such that when accessing the item $x(i)$ at time i , the expected depth of item $x(i)$ is $O(\log_B(1 + \text{interval}(i, x(i))))$. The expected total cost for processing the whole access sequence $\mathbf{X}$ is*

$$\text{cost}(\mathbf{X}, \text{isp}) = O\left(n \log_B n + \sum_{i=1}^m \log_B(1 + \text{interval}(i, x(i)))\right).$$

Furthermore, for $\delta > 0$, by setting $B = O(n^{1/2-\delta})$, the guarantees hold with probability $1 - \delta$.

Proof. We apply [Theorem 6.1](#) with $w_{i,j} = \text{isp}(i, x), i \in [m], x \in [n]$. By [Lemma 6.7](#), we know $\|w(i)\| = O(1)$, for all $i \in [m]$. Also by definition of $\text{isp}(i, x)$, for any $x \in [n]$, $\text{isp}(i-1, x) \neq \text{isp}(i, x)$ only when $x(i) = x$. So for each time i , at most one item (i.e., $x(i)$) changes its priority. So we have the total cost

$$\begin{aligned} \text{cost}(\mathbf{X}, \text{isp}) &= O\left(n \log_B n + 2 \sum_{i=1}^m \log_B(1 + \text{interval}(i, x(i))) + 4 \sum_{i=1}^m \log_B(1 + \text{interval}(i, x(i)))\right) \\ &= O\left(n \log_B n + 6 \sum_{i=1}^m \log_B(1 + \text{interval}(i, x(i)))\right) \end{aligned}$$

The last step comes from the definition of $\text{interval}(i, x)$, which is no greater than n . □

Remark. Consider two sequences with length m , $\mathbf{X}_1 = (1, 2, \dots, n, 1, 2, \dots, n, \dots, 1, 2, \dots, n)$, $\mathbf{X}_2 = (1, 1, \dots, 1, 2, 2, \dots, 2, \dots, n, n, \dots, n)$. Two sequences have the same total cost if we have a fixed score. However, $\mathbf{X}_2$ should have less cost because of its repeated pattern. Given the frequency freq as a time-invariant priority, by [Lemma 5.3](#), the optimal static costs are

$$\text{cost}(\mathbf{X}_1, \text{freq}) = \text{cost}(\mathbf{X}_2, \text{freq}) = O(m \log_B n).$$

But for the dynamic B-Trees, with the interval set priority, we calculate both costs from [Theorem 6.8](#) as

$$\text{cost}(\mathbf{X}_1, \text{isp}) = O(m \log_B(n + 1)), \text{cost}(\mathbf{X}_2, \text{isp}) = O(n \log_B n + m \log_B 3).$$

This means that our proposed priority can better capture the timing pattern of the sequence and thus can even do better than the optimal static setting.

Theorem 6.9 (Dynamic B-Treaps with Predicted Interval Set Priority). *With unknown interval set priority $\text{isp}(i, x(i)), i \in [m]$, there exists a machine learning oracle that gives their predictions $\widetilde{\text{isp}}(i, x(i)), i \in [m]$ satisfying $|\text{isp}(i, x(i)) - \widetilde{\text{isp}}(i, x(i))| > 1/\text{poly}(n)$ and the mean absolute error (MAE) is less than $\varepsilon > 0$, i.e., $\sum_{i=1}^m |\text{isp}(i, x(i)) - \widetilde{\text{isp}}(i, x(i))| \leq \varepsilon$. Let the branching factor $B = \Omega(\ln^{1/(1-\alpha)} n)$. Then there is a randomized data structure that maintains a B-Tree T^B over $[n]$ such that when accessing the item $x(i)$ at time i , the expected depth of item $x(i)$ is $O(\log_B(1 + \text{interval}(i, x(i))))$. The expected total cost for processing the whole access sequence $\mathbf{X}$ is*

$$\text{cost}(\mathbf{X}, \widetilde{\text{isp}}) = \text{cost}(\mathbf{X}, \text{isp}) + O(m \log_B(1 + n\varepsilon/m)).$$

Furthermore, for $\delta > 0$, by setting $B = O(n^{1/2-\delta})$, the guarantees hold with probability $1 - \delta$.

Proof. By the definition of interval set priority, one item will change its priority at time i only when it is accessed at time i . That is, for any $x \in [n]$, $\widetilde{\text{isp}}(i-1, x) \neq \widetilde{\text{isp}}(i, x)$ only when $x(i) = x$. In other words, at time i , we only need to learn and update $\widetilde{\text{isp}}(i, x(i))$. Denote $\mathbf{isp} = (\text{isp}(1, x(1)), \dots, \text{isp}(m, x(m)))$, $\widetilde{\mathbf{isp}} = (\widetilde{\text{isp}}(1, x(1)), \dots, \widetilde{\text{isp}}(m, x(m)))$. So we can bound the l_1 norm of $\mathbf{isp}$ as

$$\|\widetilde{\mathbf{isp}}\|_1 \leq \|\mathbf{isp}\|_1 + |\widetilde{\text{isp}}(i, x(i)) - \text{isp}(i, x(i))| = O(1)$$

The last step comes from Lemma 6.7 and the bounded MAE of the prediction. Hence we can apply Theorem 6.2 with $\widetilde{w}_{i,j} = \widetilde{\text{isp}}(i, x(i))$, $i \in [m]$, $x \in [n]$. Define $\varepsilon_{i,j} = \widetilde{\text{isp}}(i, j) - \text{isp}(i, j)$, and we get the total cost

$$\begin{aligned} \text{cost}(\mathbf{X}, \widetilde{\text{isp}}) &= O\left(n \log_B n + 2 \sum_{i=1}^m \log_B \frac{1}{w_{i,x(i)}} + 2 \sum_{i=1}^m \log_B (1 + n|\varepsilon_{i,x(i)}|)\right) \\ &= \text{cost}(\mathbf{X}, \text{isp}) + \sum_{i=1}^m \log_B (1 + n|\varepsilon_{i,x(i)}|) \\ &\leq \text{cost}(\mathbf{X}, \text{isp}) + O(m \log_B(1 + n\varepsilon/m)) \end{aligned}$$

The last step is derived from Jensen's Inequality. □

References

- [AL63] M AdelsonVelskii and Evgenii Mikhailovich Landis. *An algorithm for the organization of information*. Tech. rep. Joint Publications research service Washington DC, 1963 (cit. on p. 6).
- [AM78] Brian Allen and J. Ian Munro. “Self-Organizing Binary Search Trees”. In: *J. ACM* 25.4 (1978), pp. 526–535 (cit. on p. 6).
- [APT22] Yossi Azar, Debmalya Panigrahi, and Noam Touitou. “Online graph algorithms with predictions”. In: *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM. 2022, pp. 35–66 (cit. on p. 5).
- [AS89] Cecilia R Aragon and Raimund Seidel. “Randomized search trees”. In: *FOCS*. Vol. 30. 1989, pp. 540–545 (cit. on pp. 2, 3, 7, 8).
- [BCDI07] Mihai Bădoiu, Richard Cole, Erik D Demaine, and John Iacono. “A unified access bound on comparison-based dynamic dictionaries”. In: *Theoretical Computer Science* 382.2 (2007), pp. 86–96 (cit. on p. 6).

- [BCIKL20] Prosenjit Bose, Jean Cardinal, John Iacono, Grigorios Koumoutsos, and Stefan Langerman. “Competitive online search trees on trees”. In: *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM. 2020, pp. 1878–1891 (cit. on p. 6).
- [BEHK16] Michael A Bender, Roozbeh Ebrahimi, Haodong Hu, and Bradley C Kuszmaul. “B-trees and Cache-Oblivious B-trees with Different-Sized Atomic Keys”. In: *ACM Transactions on Database Systems (TODS)* 41.3 (2016), pp. 1–33 (cit. on p. 6).
- [BF03] Gerth Stølting Brodal and Rolf Fagerberg. “Lower bounds for external memory dictionaries.” In: *SODA*. Vol. 3. 2003, pp. 546–554 (cit. on pp. 6, 7).
- [BGVW00] Adam L Buchsbaum, Michael H Goldwasser, Suresh Venkatasubramanian, and Jeffery R Westbrook. “On external memory graph traversal.” In: *SODA*. 2000, pp. 859–860 (cit. on p. 6).
- [BR98] Guy E. Blelloch and Margaret Reid-Miller. “Fast Set Operations Using Treaps”. In: *Proceedings of the Tenth Annual ACM Symposium on Parallel Algorithms and Architectures, SPAA ’98, Puerto Vallarta, Mexico, June 28 - July 2, 1998*. Ed. by Gary L. Miller and Phillip B. Gibbons. ACM, 1998, pp. 16–26 (cit. on p. 2).
- [Bro14] Trevor Brown. “B-slack trees: Space Efficient B-trees”. In: *Scandinavian Workshop on Algorithm Theory*. Springer. 2014, pp. 122–133 (cit. on p. 7).
- [Bro17] Trevor Brown. “B-slack trees: Highly Space Efficient B-trees”. In: *arXiv preprint arXiv:1712.05020* (2017) (cit. on p. 7).
- [Can20] Clément L Canonne. “A short note on learning discrete distributions”. In: *arXiv preprint arXiv:2002.11457* (2020) (cit. on p. 14).
- [CCS19] Parinya Chalermsook, Julia Chuzhoy, and Thatchaphol Saranurak. “Pinning down the strong Wilber 1 bound for binary search trees”. In: *arXiv preprint arXiv:1912.02900* (2019) (cit. on p. 6).
- [CEILNRSWWZ22] Justin Y Chen, Talya Eden, Piotr Indyk, Honghao Lin, Shyam Narayanan, Ronitt Rubinfeld, Sandeep Silwal, Tal Wagner, David P Woodruff, and Michael Zhang. “Triangle and Four Cycle Counting with Predictions in Graph Streams”. In: *arXiv preprint arXiv:2203.09572* (2022) (cit. on p. 1).
- [CI21] Justin Y Chen and Piotr Indyk. “Online Bipartite Matching with Predicted Degrees”. In: *arXiv preprint arXiv:2110.11439* (2021) (cit. on p. 1).
- [CLRS09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009 (cit. on p. 6).
- [CM05] Graham Cormode and Shan Muthukrishnan. “An improved data stream summary: the count-min sketch and its applications”. In: *Journal of Algorithms* 55.1 (2005), pp. 58–75 (cit. on p. 5).

- [CMSS00] Richard Cole, Bud Mishra, Jeanette Schmidt, and Alan Siegel. “On the dynamic finger conjecture for splay trees. Part I: Splay sorting log n -block sequences”. In: *SIAM Journal on Computing* 30.1 (2000), pp. 1–43 (cit. on p. 6).
- [Col00] Richard Cole. “On the dynamic finger conjecture for splay trees. Part II: The proof”. In: *SIAM Journal on Computing* 30.1 (2000), pp. 44–85 (cit. on p. 6).
- [CSVZ22] Justin Chen, Sandeep Silwal, Ali Vakilian, and Fred Zhang. “Faster fundamental graph algorithms via learned predictions”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 3583–3602 (cit. on p. 1).
- [DHIKP09] Erik D Demaine, Dion Harmon, John Iacono, Daniel Kane, and Mihai Patrascu. “The geometry of binary search trees”. In: *Proceedings of the 20th annual ACM-SIAM symposium on Discrete algorithms (SODA)*. SIAM. 2009, pp. 496–505 (cit. on p. 6).
- [DHIP07] Erik D Demaine, Dion Harmon, John Iacono, and Mihai Patrascu. “Dynamic optimality—almost”. In: *SIAM Journal on Computing* 37.1 (2007), pp. 240–251 (cit. on p. 1).
- [DILMV21] Michael Dinitz, Sungjin Im, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. “Faster matchings via learned duals”. In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 10393–10406 (cit. on p. 1).
- [DMYWDLZCGK+20] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, et al. “ALEX: an updatable adaptive learned index”. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2020, pp. 969–984 (cit. on pp. 1, 5).
- [DS09] Jonathan C Derryberry and Daniel D Sleator. “Skip-splay: Toward achieving the unified bound in the BST model”. In: *Workshop on Algorithms and Data Structures*. Springer. 2009, pp. 194–205 (cit. on p. 6).
- [EFSWZ21] Jon Ergun, Zhili Feng, Sandeep Silwal, David P Woodruff, and Samson Zhou. “Learning-Augmented k -means Clustering”. In: *arXiv preprint arXiv:2110.14094* (2021) (cit. on p. 5).
- [ELMS22] Thomas Erlebach, Murilo Santos de Lima, Nicole Megow, and Jens Schlöter. “Learning-Augmented Query Policies for Minimum Spanning Tree with Uncertainty”. In: *arXiv preprint arXiv:2206.15201* (2022) (cit. on p. 1).
- [FHM19] Rolf Fagerberg, David Hammer, and Ulrich Meyer. “On Optimal Balance in B-Trees: What Does It Cost to Stay in Perfect Shape?”. In: *30th International Symposium on Algorithms and Computation (ISAAC 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2019 (cit. on p. 7).
- [FLV20] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. “Why are learned indexes so effective?”. In: *International Conference on Machine Learning*. PMLR. 2020, pp. 3123–3132 (cit. on p. 7).

- [FV20] Paolo Ferragina and Giorgio Vinciguerra. “The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds”. In: *Proceedings of the VLDB Endowment* 13.8 (2020), pp. 1162–1175 (cit. on pp. 1, 5).
- [Gol08] Daniel Golovin. “Uniquely represented data structures with applications to privacy”. PhD thesis. Carnegie Mellon University, 2008 (cit. on p. 6).
- [Gol09] Daniel Golovin. “B-treaps: A uniquely represented alternative to B-trees”. In: *Automata, Languages and Programming: 36th International Colloquium, ICALP 2009, Rhodes, Greece, July 5-12, 2009, Proceedings, Part I* 36. Springer. 2009, pp. 487–499 (cit. on pp. 2, 3, 6, 12).
- [GS78] Leo J Guibas and Robert Sedgewick. “A dichromatic framework for balanced trees”. In: *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. IEEE. 1978, pp. 8–21 (cit. on p. 6).
- [HIKV19] Chen-Yu Hsu, Piotr Indyk, Dina Katabi, and Ali Vakilian. “Learning-Based Frequency Estimation Algorithms.” In: *International Conference on Learning Representations*. 2019 (cit. on pp. 5, 7).
- [HT70] TC Hu and AC Tucker. *Optimum Binary Search Trees*. Tech. rep. WISCONSIN UNIV MADISON MATHEMATICS RESEARCH CENTER, 1970 (cit. on p. 6).
- [Iac01] John Iacono. “Alternatives to splay trees with $O(\log n)$ worst-case access times”. In: *Proceedings of the twelfth annual ACM-SIAM symposium on Discrete algorithms*. 2001, pp. 516–522 (cit. on p. 6).
- [Iac05] John Iacono. “Key-independent optimality”. In: *Algorithmica* 42.1 (2005), pp. 3–10 (cit. on p. 6).
- [Iac13] John Iacono. “In pursuit of the dynamic optimality conjecture”. In: *Space-Efficient Data Structures, Streams, and Algorithms*. Springer, 2013, pp. 236–250 (cit. on p. 6).
- [JDO99] Chris Jermaine, Anindya Datta, and Edward Omiecinski. “A novel index supporting high volume data warehouse insertion”. In: *VLDB*. Vol. 99. 1999, pp. 235–246 (cit. on p. 6).
- [JLLRW20] Tanqiu Jiang, Yi Li, Honghao Lin, Yisong Ruan, and David P Woodruff. “Learning-augmented data stream algorithms”. In: *ICLR* (2020) (cit. on p. 5).
- [JNSSK97] HV Jagadish, PPS Narayan, Sridhar Seshadri, S Sudarshan, and Rama Kanneganti. “Incremental organization for data recording and warehousing”. In: *VLDB*. 1997, pp. 16–25 (cit. on p. 6).
- [KBCDP18] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. “The case for learned index structures”. In: *Proceedings of the 2018 international conference on management of data*. 2018, pp. 489–504 (cit. on pp. 1, 5).
- [KLR96] Marek Karpinski, Lawrence L Larmore, and Wojciech Rytter. “Sequential and Parallel Subquadratic Work Algorithms for Constructing Approximately Optimal Binary Search Trees.” In: *SODA*. Citeseer. 1996, pp. 36–41 (cit. on p. 6).

- [LLW22] Honghao Lin, Tian Luo, and David Woodruff. “Learning Augmented Binary Search Trees”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 13431–13440 (cit. on pp. 2, 3, 7, 11).
- [LMRX20] Thomas Lavastida, Benjamin Moseley, R Ravi, and Chenyang Xu. “Learnable and instance-robust predictions for online matching, flows and load balancing”. In: *arXiv preprint arXiv:2011.11743* (2020) (cit. on p. 1).
- [MA13] Giorgos Margaritis and Stergios V Anastasiadis. “Efficient range-based storage management for scalable datastores”. In: *IEEE Transactions on Parallel and Distributed Systems* 25.11 (2013), pp. 2851–2866 (cit. on p. 7).
- [Meh75a] Kurt Mehlhorn. “Best possible bounds for the weighted path length of optimum binary search trees”. In: *Automata Theory and Formal Languages, 2nd GI Conference, Kaiserslautern, May 20-23, 1975*. Ed. by H. Barkhage. Vol. 33. Lecture Notes in Computer Science. Springer, 1975, pp. 31–41 (cit. on p. 1).
- [Meh75b] Kurt Mehlhorn. “Nearly optimal binary search trees”. In: *Acta Informatica* 5.4 (1975), pp. 287–295 (cit. on p. 6).
- [MV20] Michael Mitzenmacher and Sergei Vassilvitskii. “Algorithms with predictions”. In: *arXiv preprint arXiv:2006.09123* (2020) (cit. on pp. 1, 5).
- [OCGO96] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. “The log-structured merge-tree (LSM-tree)”. In: *Acta Informatica* 33 (1996), pp. 351–385 (cit. on p. 6).
- [PZ22] Adam Polak and Maksym Zub. “Learning-Augmented Maximum Flow”. In: *arXiv preprint arXiv:2207.12911* (2022) (cit. on p. 1).
- [RS81] Arnold L Rosenberg and Lawrence Snyder. “Time-and space-optimality in B-trees”. In: *ACM Transactions on Database Systems (TODS)* 6.1 (1981), pp. 174–193 (cit. on p. 6).
- [SS23] Roodabeh Safavi and Martin P Seybold. “B-Treaps Revised: Write Efficient Randomized Block Search Trees with High Load”. In: *arXiv preprint arXiv:2303.04722* (2023) (cit. on p. 7).
- [ST85a] Daniel Dominic Sleator and Robert Endre Tarjan. “Self-Adjusting Binary Search Trees”. In: *J. ACM* 32.3 (1985), pp. 652–686 (cit. on p. 15).
- [ST85b] Daniel Dominic Sleator and Robert Endre Tarjan. “Self-adjusting binary search trees”. In: *Journal of the ACM (JACM)* 32.3 (1985), pp. 652–686 (cit. on pp. 1, 6).
- [Vit01] Jeffrey Scott Vitter. “External memory algorithms and data structures: Dealing with massive data”. In: *ACM Computing surveys (CsUR)* 33.2 (2001), pp. 209–271 (cit. on p. 7).
- [WZCWCX21] Jiacheng Wu, Yong Zhang, Shimin Chen, Jin Wang, Yu Chen, and Chunxiao Xing. “Updatable learned index with precise positions”. In: *Proceedings of the VLDB Endowment* 14.8 (2021), pp. 1276–1288 (cit. on p. 5).
- [Yao82] F Frances Yao. “Speed-up in dynamic programming”. In: *SIAM Journal on Algebraic Discrete Methods* 3.4 (1982), pp. 532–540 (cit. on p. 6).

- [Yi12] Ke Yi. “Dynamic Indexability and the Optimality of B-trees”. In: *Journal of the ACM (JACM)* 59.4 (2012), pp. 1–19 (cit. on p. [6](#)).
- [ZB22] Eva Zangerle and Christine Bauer. “Evaluating Recommender Systems: Survey and Framework”. In: 55.8 (Dec. 2022) (cit. on p. [5](#)).